

Shielding으로 안전한 강화학습 만들기

Safe Reinforcement Learning via Shielding

Mohammed Alshiekh, Roderick Bloem, Rüdiger Ehlers, Bettina Könighofer, Scott Niekum, and
Ufuk Topcu (2017)

보상 최적화와 시간 논리 안전 조건을 분리하고 다시 연결하는 방법

높은 reward가 안전을 뜻하지는 않는다

위험에 큰 penalty를 주어도 탐색 중 사고가 먼저 일어날 수 있다

- | | | |
|----|---------------|--------------------------|
| 01 | Learner | 미지의 환경에서 action을 시험 |
| ↓ | | |
| 02 | Environment | 다음 state와 reward를 반환 |
| ↓ | | |
| 03 | Policy update | 장기 return이 커지도록 수정 |
| ↓ | | |
| 04 | 남는 문제 | 학습·실행 중 안전 위반을 막는 보장은 없음 |

Reward 최적화와 매 순간의 안전 보장은 서로 다른 요구사항이다.

두 역할을 분리한다

구성요소	말는 목표	입력	출력
Learning agent	성능 기준의 return 최적화	State·reward	선호 action 또는 ranking
Shield	Safety specification 준수	관찰 label·action	안전 action 집합 또는 실행 action
Environment	Dynamics와 reward 제공	실제로 실행된 action	다음 state·reward

Learner는 성능을 배우고 shield는 허용 가능한 행동 경계를 지킨다.

먼저 구분할 네 가지 대상

기호·대상	뜻	Shield가 사용하는 부분
MDP M	실제 state, action, transition, reward	직접 완전히 알 필요는 없음
Observer f	State를 유한 label로 바꾸는 함수	안전에 관련된 관찰만 전달
Safety automaton φ s	허용 trace를 safe state로 표현	나쁜 prefix가 생기기 전 차단
Abstraction φ M	가능한 환경 label 변화를 보수적으로 표현	앞으로 생길 수 있는 경우를 계산

Shield는 원시 state 전체보다 안전 판단에 필요한 label과 추상 동작을 본다.

Safety는 유한한 bad prefix로 드러난다

한 번 생기면 이후 행동으로 되돌릴 수 없는 위반의 시작을 추적한다

01 Trace 관찰 label과 action이 시간 순서로 쌓임



02 Automaton state 지금까지의 안전 관련 history를 요약



03 Unsafe transition 위반이 확정되는 bad prefix가 생성



04 Shield 그 전 단계에서 해당 action을 차단

Shield는 사고 뒤 복구가 아니라 위반이 확정되기 전의 action 제한을 목표로 한다.

Safety automaton이 기억하는 것

현재 state q 는 지금까지의 trace에서 앞으로 필요한 제약 정보를 압축한다

구성	역할	읽는 값
Q, q_0	유한 state와 초기 state	안전 관련 history의 요약
$\Sigma = L \times A$	관찰 label과 action의 alphabet	현재 label과 선택 action
δ	다음 automaton state 결정	q , label, action
F	계속 머물러야 하는 safe states	Run 전체가 F 안인지 검사

현재 관찰만으로 부족한 시간 조건은 automaton state가 보완한다.

Shield 합성에는 두 입력이 필요하다



Safety game은 환경과 시스템이 번갈아 움직인다



Winning action은 가능한 환경 변화까지 고려한 뒤에도 W 안에 남게 하는 선택이다.

Winning region W의 의미

구분	판정	Shield의 해석
$g \in W$	환경의 모든 허용 선택에 대응 가능	안전 전략을 계속 이어갈 수 있음
$g \notin W$	환경이 장치 위반을 강제할 수 있음	그 state로 가는 action을 허용하지 않음
Reachable winning state	최소 한 safe action이 존재	Preemptive action set이 비지 않음
Abstraction error	실제 trace가 추상화 밖으로 나감	논문의 보장 조건이 깨졌다는 신호

W는 지금 안전한 state가 아니라 앞으로도 안전을 강제할 수 있는 state 집합이다.

왜 abstraction은 보수적이어야 하는가

- | | | |
|----|--------------|-------------------------------|
| 01 | 실제 dynamics | 여러 next state가 확률적으로 가능 |
| ↓ | | |
| 02 | Labeling | 안전 판단에 필요한 유한 label로 축약 |
| ↓ | | |
| 03 | Abstraction | 가능한 label transition을 빠뜨리지 않음 |
| ↓ | | |
| 04 | Game solving | 그 모든 경우에 안전한 action만 남김 |

가능한 위험 전이를 누락하면 shield가 실제 위험을 안전하다고 오판할 수 있다.

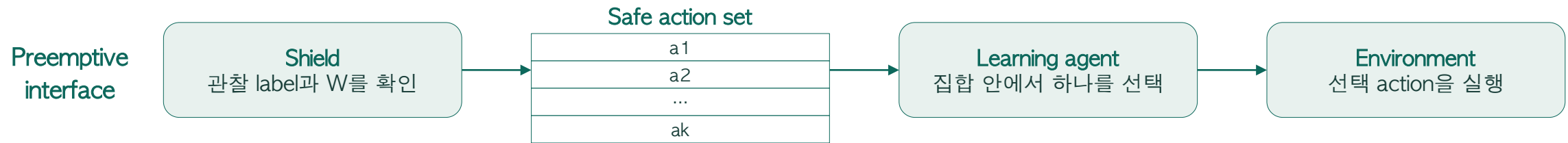
Water tank에서 미리 막아야 하는 이유

밸브를 바꾸면 최소 세 단계 유지해야 하므로 현재 수위만 보고 결정할 수 없다

상황	가능한 변화	Shield 판단
Tank level이 너무 낮음	Outflow가 계속될 수 있음	Inflow를 켜고 최소 level까지 유지
Level이 93 초과	Inflow가 초당 최대 2 L, outflow는 0 가능	밸브를 새로 열지 않음
Valve 변경 직후	최소 3초 동안 mode 유지	미래 level 범위를 함께 계산

Shield는 즉시 위반뿐 아니라 의무적으로 이어질 행동의 최악 경우까지 본다.

Preemptive shield: 선택 전에 후보를 줄인다



Learner는 처음부터 안전 action 집합 바깥을 선택할 수 없다.

Post-posed shield: 제안 뒤 실행 action을 정한다



제안 action이 unsafe일 때만 shield가 safe action으로 교체한다.

두 interface의 차이

항목	Preemptive	Post-posed
Shield 위치	Learner의 선택 전	Learner의 제안 후
Shield 출력	Safe action들의 집합	실제로 실행할 action 하나
Learner 입력	Product state와 제한 action set	원래 state만으로도 동작 가능
Unsafe proposal	애초에 선택 불가	Safe action으로 교정
Deployment	제한된 action space 유지	Fixed policy에도 덧붙일 수 있음

두 방식은 같은 W 를 사용하지만 learner와 shield 사이의 계약이 다르다.

교육용 예시: go와 brake를 W 로 판정한다

논문 실험값이 아닌 interface 계산 예시이며 현재 game state g 와 label near를 가정한다

후보 action	다음 game state	W 포함 여부	판정
go	$\delta(g, \text{near}, \text{go}) = g_{\text{bad}}$	$g_{\text{bad}} \notin W$	Unsafe
brake	$\delta(g, \text{near}, \text{brake}) = g_{\text{safe}}$	$g_{\text{safe}} \in W$	Safe

이 예시의 safe action set은 정확히 $\{\text{brake}\}$ 다.

교육용 예시의 preemptive 실행

01	관찰	현재 g, label near
↓		
02	Filter	go 제거, brake 유지
↓		
03	Learner input	Safe set {brake}
↓		
04	실행	Learner가 brake를 선택

Preemptive 방식에서는 unsafe action이 learner의 후보 목록에 나타나지 않는다.

교육용 예시의 post-posed 실행

- | | | |
|----|-----------------|-------------------------|
| 01 | Learner ranking | [go, brake]를 제안 |
| ↓ | | |
| 02 | 1순위 검사 | go는 successor가 W 밖 |
| ↓ | | |
| 03 | 2순위 검사 | brake는 successor가 W 안 |
| ↓ | | |
| 04 | 실행 | 첫 safe action brake를 전달 |

Ranking을 쓰면 shield는 learner의 선호를 가능한 범위에서 최대한 따른다.

교정된 action을 어떻게 학습에 반영하는가

방법	Unsafe proposal 처리	중요한 경계
Punishment variant	제안한 unsafe action에 음의 reward 부여	Final policy에서 unsafe action이 사라진다는 보장은 없음
No-punishment variant	실행된 safe action의 reward를 proposal에도 사용	Unsafe proposal이 optimal policy에 남을 수 있음
공통	실제 실행 action과 reward를 learner에 전달	Learning과 deployment 모두 shield가 필요할 수 있음

Shield가 안전을 담당하므로 learner가 unsafe action을 완전히 잊는다고 가정하지 않는다.

Shield 합성 절차

01	1. Automata	Safety specification과 MDP abstraction 준비
↓		
02	2. Product	State space $G = Q_{\text{spec}} \times Q_{\text{abs}}$ 구성
↓		
03	3. Game solve	Safety winning region W 계산
↓		
04	4. Interface	Action set 또는 교정 action 출력
↓		
05	5. Runtime	Observer label과 action으로 shield state 갱신

두 shielding 방식은 product game과 W 계산까지 같은 합성 절차를 공유한다.

Realizable해야 safe action이 계속 존재한다

조건	성립할 때	성립하지 않을 때
초기 game state가 W 에 있음	Winning strategy로 시작 가능	처음부터 안전 강제가 불가능
환경이 abstraction을 따름	Game이 고려한 transition 안에서 전개	관찰된 trace가 모델 밖으로 나감
Shield가 W 안을 유지	매 step 최소 한 action 존재	Losing state에 들어가면 보장 불가

명세가 존재한다는 것만으로 충분하지 않고 주어진 환경에서 강제 가능한 명세여야 한다.

정확성 보장은 조건부다

보장하는 것	필요 조건	보장하지 않는 것
Learning과 execution trace가 safety specification을 만족	실제 환경이 abstraction에 부합하고 초기 state가 winning	Shield 밖의 행동이나 누락된 hazard
Preemptive safe action set이 비지 않음	Reachable shield state가 W 안에 유지	모든 action을 허용한다는 의미
오류 state 진입은 abstraction 오류 뒤에만 가능	Observer와 automata update가 정의대로 동작	Abstraction 자체의 정확성 자동 검증

Shield의 formal guarantee는 specification과 abstraction이 덮는 동작에 한정된다.

Minimal interference의 정확한 의미

질문	논문의 답	오해하면 안 되는 것
왜 action을 막는가	Abstraction과 일치하는 어떤 MDP에서 위반 확률이 0보다 클 수 있음	현재 step에서 즉시 충돌한다는 뜻만은 아님
이후 policy가 잘하면 되는가	막힌 action 뒤에는 어떤 후속 policy라도 위험을 피하지 못하는 경우가 존재	평균적으로 안전하면 허용한다는 기준이 아님
최소라는 뜻	모든 일치 MDP에서 안전을 보장하는 shield 중 덜 제한적	Reward loss나 regret의 최소값이 아님

Minimal interference는 안전 제약의 최소성이기 성능 손실의 최적화가 아니다.

Convergence는 product MDP로 옮겨 생각한다

항목	필요 조건	결론의 범위
State	Discrete MDP와 shield state의 product를 learner가 구분	표준 MDP로 표현 가능
Preemptive	State별 action set이 shield로 제한됨	그 product MDP에서의 convergence
Post-posed	Fallback safe-action 분포가 시간에 따라 변하지 않음	Joint dynamics가 stationary MDP
Learning algorithm	표준 MDP에서 원래 converge하는 알고리즘	같은 종류의 convergence 이전

논문은 모든 neural RL의 수렴을 새로 증명한 것이 아니다.

안전 보장과 학습 보장을 분리해서 읽는다

주장	논문이 제공	제공하지 않음
Safety	명세·추상화 범위에서 runtime trace 보호	명세 밖의 모든 현실 위험 제거
Optimality	Shielded product MDP 안에서 learner가 최적화	원래 unconstrained MDP의 global optimum
Exploration	Unsafe action 실행을 차단	모든 safe state를 충분히 방문
Generalization	Observer가 구분한 label에 대응	학습하지 않은 state에서 neural policy 정확성
Model validity	Abstraction을 입력으로 사용	Abstraction 오류의 자동 탐지·복구

Formal safety theorem을 성능·탐색·미지 state 보장으로 확장해서 읽으면 안 된다.

실험 1 : Grid world

Tabular Q-learning과 epsilon-greedy exploration, 9×9 및 15×9 환경

조건	9×9	15×9
Task	색 영역을 순서대로 방문	색 영역을 순서대로 방문
Safety	벽 회피 + bomb 위 2단계 초과 금지	벽과 이동 opponent 충돌 회피
Shield synthesis	정밀 abstraction, 2 s	정밀 abstraction, 0.6 s
관찰 결과	Unshielded만 음의 reward 경험	Ranking 3·no penalty만 optimal path 발견

이 결과는 두 grid 설정과 비교 variant에 조건부이며 보편적 속도 향상 증거는 아니다.

실험 2: Self-driving car simulation

항목	논문의 조건	관찰 결과
Environment	480 × 480 pixels, step당 3 pixels 이동	시계 방향 주행 reward
Action·turn	8 heading outputs, 최대 7.5° 회전	벽 충돌 시 penalty와 restart
Shield observation	차량 측면이 벽에서 60 pixels 이내인지	Preemptive·post-posed 모두 2 s 합성
Learner	DQN, Boltzmann exploration, 3 hidden layers	No-punishment shield가 더 빨리 학습하고 crash 없음

Crash-free 결과는 이 simulator와 관찰 interface에서 보고된 결과다.

실험 3: Atari Seaquest

구성	Learner	Shield	결과
입력	화면 RGB image	Simulator state에서 proposition 계산	서로 다른 관찰 경로
Safety 1	Score 최대화	Oxygen 소진 전 수면 도달	위반 없이 유지
Safety 2	내부 변경 없는 black box	산소 충분·diver 없음이면 수면 금지	성능 plot은 유사
Interface	Action 제안	Unsafe action만 교정	Shielded performance 저하 없음

Shield는 learner의 RGB 입력과 별개인 simulator proposition을 사용했다.

실험 4: Water tank

항목	조건	결과
Safety	Dry·overflow 방지, mode 변경 뒤 3단계 유지	Post-posed shield가 제약 강제
Concrete MDP	State-dependent energy, 여러 local minima	Q-learning과 SARSA 비교
Shield synthesis	Specification 두 개와 abstraction 사용	1 s 미만
Learning curve	Shielded·unshielded 모두 optimum 도달	Shielded가 더 적은 episode에 optimum 도달

이 환경에서는 안전 제한이 탐색을 줄여 최적 policy 도달을 앞당겼다.

네 실험을 같은 기준으로 읽기

Domain	Learner	Shield에 필요한 정보	주된 관찰
Grid	Tabular Q-learning	정밀 grid abstraction	위반 penalty 회피와 경로 차이
Car	DQN	벽까지 60-pixel proximity	Simulation crash 방지
Seaquest	RGB DQN black box	Simulator propositions	성능 유지와 명세 위반 방지
Water tank	Q-learning-SARSA	Tank dynamics abstraction	Optimum 도달 episode 감소

공통점은 learner 종류가 아니라 안전 관찰과 abstraction을 별도로 제공했다는 점이다.

적용 전에 확인할 한계

한계	왜 중요한가	실무 질문
Abstraction 필요	위험 action의 미래 결과를 알아야 함	가능한 hazard transition을 빠뜨렸는가
Specification coverage	적지 않은 위험은 shield가 알 수 없음	현실의 안전 요구가 automaton에 들어갔는가
Observer 신뢰성	잘못된 label은 잘못된 game state를 만듦	Sensor-perception error를 어떻게 다루는가
Conservatism	과도한 transition은 safe action을 줄임	안전성과 행동 자유도의 균형은 어떤가
Runtime dependency	Learned policy가 unsafe proposal을 유지할 수 있음	배포 중 shield를 계속 운영할 수 있는가

Shielding은 model-free safety가 아니라 행동 경계의 formal model이다.

핵심 흐름을 한 번 더 연결하면

01	Specify	Safety condition을 automaton으로 표현
↓		
02	Abstract	가능한 환경 변화를 보수적으로 표현
↓		
03	Synthesize	Product game에서 winning region 계산
↓		
04	Shield	선택 전 filter 또는 선택 후 override
↓		
05	Learn	안전 범위 안에서 reward를 최적화

Shielding은 안전과 학습을 분리해 실행 경계에서 결합한다.